

Day 6 — 工程化实战（完整版）

一、项目结构 — 从一开始就做对

1. 是什么

Python 项目没有"官方"目录结构。但社区总结出了最佳实践——**src** 布局。

```
my-project/
├─ src/
│   └─ my_package/
│       ├── __init__.py      # 包标记 + 公共导出
│       ├── __main__.py      # python -m my_package 入口
│       ├── cli.py
│       ├── models.py
│       └─ utils.py
├─ tests/
│   ├── __init__.py
│   ├── test_models.py
│   └─ test_cli.py
├─ pyproject.toml            # 现代包配置入口
├─ README.md
├─ .gitignore
└─ Dockerfile
```

2. 解决了什么问题

不用 src 布局，测试时可能 import 了错误路径：

```
# 项目根目录下直接放 my_package/
# 测试时 import my_package 可能 import 了项目目录（没安装的版本）
# 而不是 pip install 好的正式版本
```

```
# src 布局强迫你先 pip install -e . 安装包
# 测试环境和用户环境一致 → 不会再出现"我机器上能跑"
```

3. pyproject.toml

现代 Python 包配置——取代了 setup.py、setup.cfg 和 requirements.txt。

```
[build-system]
requires = ["setuptools>=68.0"]
build-backend = "setuptools.build_meta"

[project]
name = "my-cli-tool"
version = "0.1.0"
description = "一个有用的 CLI 工具"
requires-python = ">=3.10"
dependencies = [
    "requests>=2.31",
    "click>=8.1",
]

[project.optional-dependencies]
dev = [
    "pytest>=8.0",
    "ruff>=0.5",
    "mypy>=1.10",
]

[project.scripts]
my-cli = "my_package.cli:main" # 安装后命令行直接敲 my-cli

[tool.setuptools.packages.find]
where = ["src"]

[tool.ruff]
line-length = 100
```

```
pip install -e .          # 开发模式安装（可编辑）
pip install -e ".[dev]"   # 带开发依赖
```

文件	作用	还常见吗
<code>pyproject.toml</code>	一切配置的入口	✓ 标准
<code>setup.py</code>	复杂构建逻辑	⚠ 部分项目还有
<code>setup.cfg</code>	旧式声明式配置	✗ 已被 toml 取代
<code>requirements.txt</code>	精确锁定依赖版本	✓ 部署用

二、pytest — 专业测试框架

1. 是什么

pytest 是 Python 最流行的测试框架。核心哲学：用断言代替样板代码。

特性	unittest	pytest
测试用例	必须继承 <code>TestCase</code>	普通函数就行
断言	<code>self.assertEqual(a, b)</code>	<code>assert a == b</code>
fixture	<code>setUp / tearDown</code>	<code>@pytest.fixture</code>
参数化	需要子类	<code>@pytest.mark.parametrize</code>
插件生态	一般	丰富（ <code>pytest-cov</code> , <code>pytest-mock</code> ）

2. 对比

```
# unittest 方式
import unittest
class TestMath(unittest.TestCase):
```

```

    def test_add(self):
        self.assertEqual(1 + 1, 2)

# pytest 方式 – 就是普通函数
def test_add():
    assert 1 + 1 == 2

```

3. 核心 API

```

pytest                                # 自动发现 test_*.py / *_test.py
pytest -v                             # 详细输出
pytest -x                             # 第一个失败就停
pytest --pdb                          # 失败进调试器
pytest -k "add or dict"               # 按名字过滤
pytest test_math.py::test_add         # 只跑指定测试
pytest -s                             # 显示 print 输出
pytest --cov=my_package               # 覆盖率报告
pytest -v --durations=5               # 显示最慢的 5 个测试
pytest --lf                           # 只跑上次失败的
pytest --ff                           # 先跑上次失败的，再跑剩下的

```

Fixture

```

import pytest

@pytest.fixture
def temp_dir():
    """创建临时目录，测试完自动清理"""
    import tempfile
    with tempfile.TemporaryDirectory() as tmp:
        yield tmp # yield 之前的代码 = setUp, 之后的 = tearDown

@pytest.fixture
def sample_data():
    """返回测试数据（无副作用）"""
    return {"name": "Leo", "age": 30}

```

```

# fixture 作用域
@pytest.fixture(scope="session")    # 一次测试会话只创建一次
@pytest.fixture(scope="module")     # 每个模块一次
@pytest.fixture(scope="class")      # 每个类一次
@pytest.fixture(scope="function")   # 每个测试函数一次（默认）

def test_file_creation(temp_dir):
    path = os.path.join(temp_dir, "test.txt")
    with open(path, "w") as f:
        f.write("hello")
    assert os.path.exists(path)

```

参数化测试

```

@pytest.mark.parametrize("input,expected", [
    (1, 1),
    (2, 4),
    (3, 9),
    (10, 100),
])
def test_square(input, expected):
    assert input ** 2 == expected

# 多参数组合（笛卡尔积）
@pytest.mark.parametrize("x", [1, 2])
@pytest.mark.parametrize("y", [10, 20])
def test_combine(x, y):
    assert x + y > 0    # 跑 4 次：(1,10), (1,20), (2,10), (2,20)

```

Mock

```

from unittest.mock import Mock, patch

# Mock 一个对象
mock_response = Mock()
mock_response.json.return_value = {"status": "ok"}
mock_response.status_code = 200

```

```
# patch 替换真实函数
with patch("requests.get", return_value=mock_response):
    result = my_function()
    assert result["status"] == "ok"

# patch 也可以作为装饰器
@patch("requests.get")
def test_api(mock_get):
    mock_get.return_value.json.return_value = {"success": True}
    result = call_api()
    assert result["success"]
```

conftest.py

```
# tests/conftest.py - 自动被所有测试文件共享
import pytest
from my_package import create_app

@pytest.fixture
def app():
    app = create_app()
    yield app

@pytest.fixture
def client(app):
    return app.test_client()

# tests/test_routes.py - 直接用
def test_home(client):
    resp = client.get("/")
    assert resp.status_code == 200
```

三、CLI 工具 — Click / Typer

1. 是什么

Click 是 Python 最成熟的命令行工具库。

2. 解决了什么问题

不用再手写 `sys.argv` 解析。

```
# ✖ 手工解析
if len(sys.argv) < 2:
    print("Usage: ...")
    sys.exit(1)
name = sys.argv[1]

# ✔ Click
import click

@click.command()
@click.argument("name")
@click.option("--count", default=1, help="问候次数")
@click.option("--upper/--no-upper", default=False)
@click.option("--greeting", default="Hello", show_default=True)
def greet(name, count, upper, greeting):
    """问候某人。"""
    for _ in range(count):
        msg = f"{greeting} {name}"
        if upper:
            msg = msg.upper()
        click.echo(msg)

if __name__ == "__main__":
    greet()
```

```
python greet.py Leo --count 3 --upper
python greet.py --help
```

```
# 自动生成：
# Usage: greet.py [OPTIONS] NAME
```

多命令分组

```
@click.group()
def cli():
    """项目管理工具"""
    pass

@cli.command()
def init():
    """初始化项目"""
    click.echo("项目已初始化 ✓")

@cli.command()
@click.argument("name")
def create(name):
    """创建新项目"""
    click.echo(f"已创建 {name} ✓")

@cli.command()
@click.option("--force", is_flag=True)
def clean(force):
    """清理项目"""
    if force:
        click.confirm("确认删除?", abort=True)
    click.echo("已清理 ✓")

if __name__ == "__main__":
    cli()
```

```
python cli.py init
python cli.py create my-app
python cli.py clean --force
python cli.py --help # 自动列出所有命令
```


Click 特性	写法	效果
Argument	<code>@click.argument("name")</code>	必填位置参数
Option	<code>@click.option("--count")</code>	可选命名参数
Flag	<code>@click.option("--verbose/--quiet")</code>	布尔开关
确认	<code>click.confirm("确认?")</code>	交互确认
选择	<code>click.Choice(["dev", "prod"])</code>	限制可选值
密码	<code>click.prompt("Password", hide_input=True)</code>	隐藏输入

四、FastAPI 入门

1. 是什么

FastAPI 是目前 Python 增长最快的 Web 框架。核心卖点：基于类型注解的自动化。

2. 解决了什么问题

方法签名、数据校验、文档三件事应该同步，而不是各写各的。

```
# Flask 方式：手写校验，手写文档
@app.route("/items/<int:item_id>")
def get_item(item_id):
    if not isinstance(item_id, int):
        return {"error": "id must be int"}, 400

# FastAPI 方式：类型注解 = 校验 + 文档
@app.get("/items/{item_id}")
def get_item(item_id: int): # 自动校验类型，生成 OpenAPI 文档
    ...
```

3. 核心理论

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

app = FastAPI(title="我的 API")

class Item(BaseModel):
    name: str
    price: float
    in_stock: bool = True

items: dict[int, Item] = {}
next_id = 1

@app.post("/items", status_code=201)
def create_item(item: Item):
    global next_id
    items[next_id] = item
    item_id = next_id
    next_id += 1
    return {"id": item_id, **item.model_dump()}

@app.get("/items/{item_id}")
def get_item(item_id: int):
    if item_id not in items:
        raise HTTPException(404, "物品未找到")
    return {"id": item_id, **items[item_id].model_dump()}

@app.get("/items")
def list_items(skip: int = 0, limit: int = 10):
    return list(items.values())[skip:skip + limit]
```

```
pip install fastapi uvicorn
uvicorn app:app --reload
# http://localhost:8000/docs - 自动 Swagger 文档！
```

4. Pydantic 进阶

```
from pydantic import BaseModel, Field, HttpUrl, EmailStr

class User(BaseModel):
    name: str = Field(min_length=1, max_length=100)
    email: str
    age: int = Field(ge=0, le=150)
    website: HttpUrl | None = None

    @field_validator("email")
    @classmethod
    def email_must_contain_at(cls, v):
        if "@" not in v:
            raise ValueError("邮箱必须包含 @")
        return v.lower()
```

5. 依赖注入

```
from fastapi import Depends, Header

def get_db():
    db = Database()
    try:
        yield db
    finally:
        db.close()

def verify_token(token: str = Header(...)):
    if token != "my-secret":
        raise HTTPException(401)

@app.get("/items")
def list_items(
    db = Depends(get_db),
    token = Depends(verify_token),
):
    return db.query_all()
```

FastAPI 特性	作用	代码量节省
类型注解 = 校验	自动验证请求参数	50%
类型注解 = 文档	自动 OpenAPI + Swagger UI	100%
Pydantic	请求/响应体校验	60%
Depends	依赖注入	40%
自动序列化	dict → JSON	30%

五、SQLAlchemy — 数据库 ORM

1. 是什么

SQLAlchemy 是 Python 最强大的 ORM（对象关系映射）。2.0 版本大幅简化了 API。

2. 解决了什么问题

```
# ✖ 手写 SQL
cursor.execute("SELECT * FROM items WHERE price > ?", (500,))
rows = cursor.fetchall()
for row in rows:
    print(f"Item: {row[0]}, Price: {row[3]}") # 魔法下标

# ✔ SQLAlchemy: 操作 Python 对象
items = session.query(Item).filter(Item.price > 500).all()
for item in items:
    print(f"Item: {item.name}, Price: {item.price}") # 属性名
```

3. 核心理论

```
from sqlalchemy import create_engine, Column, Integer, String, Float,
DateTime
from sqlalchemy.orm import declarative_base, Session
```

```
engine = create_engine("sqlite:///items.db", echo=True)
Base = declarative_base()

class Item(Base):
    __tablename__ = "items"
    id = Column(Integer, primary_key=True)
    name = Column(String, nullable=False)
    price = Column(Float, default=0.0)
    created_at = Column(DateTime)

Base.metadata.create_all(engine)  # 自动建表

# Create
with Session(engine) as session:
    item = Item(name="笔记本电脑", price=999.99)
    session.add(item)
    session.commit()
    session.refresh(item)
    print(item.id)

# Read
item = session.get(Item, 1)
expensive = session.query(Item).filter(Item.price > 500).all()
cheap = session.query(Item).filter_by(price=0).all()

# 高级查询
from sqlalchemy import desc, func
session.query(Item).filter(
    Item.name.like("%笔记本%")
).order_by(desc(Item.price)).limit(10).all()

# Update
item = session.get(Item, 1)
item.price = 899.99
session.commit()

# Delete
session.delete(item)
session.commit()
```

4. 关联查询

```
from sqlalchemy import ForeignKey
from sqlalchemy.orm import relationship

class User(Base):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True)
    name = Column(String)
    items = relationship("Item", back_populates="owner")

class Item(Base):
    __tablename__ = "items"
    id = Column(Integer, primary_key=True)
    name = Column(String)
    owner_id = Column(Integer, ForeignKey("users.id"))
    owner = relationship("User", back_populates="items")

# 关联查询
user = session.get(User, 1)
print(user.items)          # 自动关联
item = session.get(Item, 1)
print(item.owner.name)     # 反向关联
```

今日练习

1. 用 pyproject.toml + src 布局创建新项目，配好 pytest 和 ruff
2. 用 Click 写一个 TODO 管理工具（add, list, done, delete），数据存 JSON
3. 用 FastAPI 把 TODO 工具变成 REST API，Pydantic 做校验
4. 用 pytest（fixture + parametrize + mock）给 TODO API 写完整测试

明天预告：Day 7 — 实战项目（从零搭建 URL 短链服务，综合所有知识）